

AdventureWorks Data Engineering Project

Bronze to Gold implementation report

Prepared by Nandini Babu

This report documents the AdventureWorks work shown in the supplied Azure and Databricks screenshots. The implemented flow lands ten CSV source files in Azure Data Lake Storage, transforms them into Silver Parquet outputs, and writes six Gold Delta datasets for analysis. The report ends after Gold validation and storage. The accompanying project guide describes additional semantic and dashboard stages, which are outside this report.

Project scope and architecture

Stage	Observed input and output	Purpose
Bronze	Ten source CSV files in the bronze container	Keep source data available for reading and replay.
Silver	Transformed Parquet folders in the silver container	Standardize fields, dates, names and sales records.
Gold	Four dimensions and two facts saved as Delta folders	Prepare modeled data with sales measures and relationships.

Evidence note: The project guide recommends Delta format in Bronze and Silver. The supplied screenshots show CSV files in Bronze and Parquet files in Silver. Gold is written with format("delta"). This report follows the screenshots when describing the implementation.

1 Azure environment and storage

An application registration and Azure role assignment were used while configuring access for Databricks. The screenshots show the application and role selection; they do not independently prove every permission was effective at all times. The storage account contains bronze, silver and gold containers.

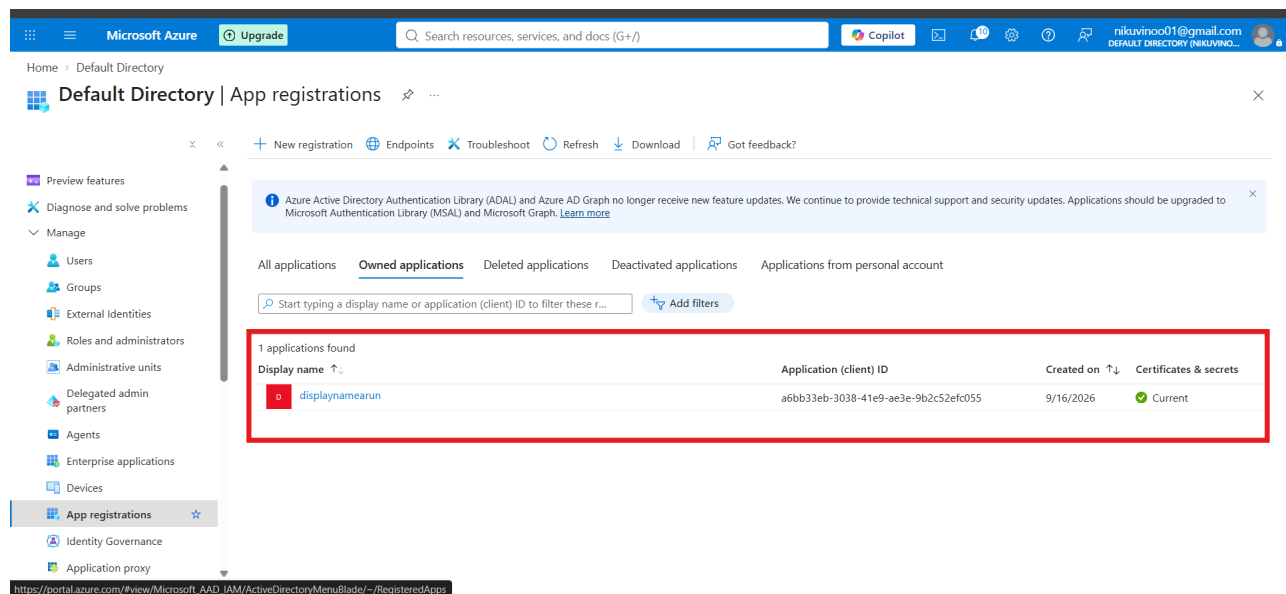


Figure 1 Azure application registration used in the setup.

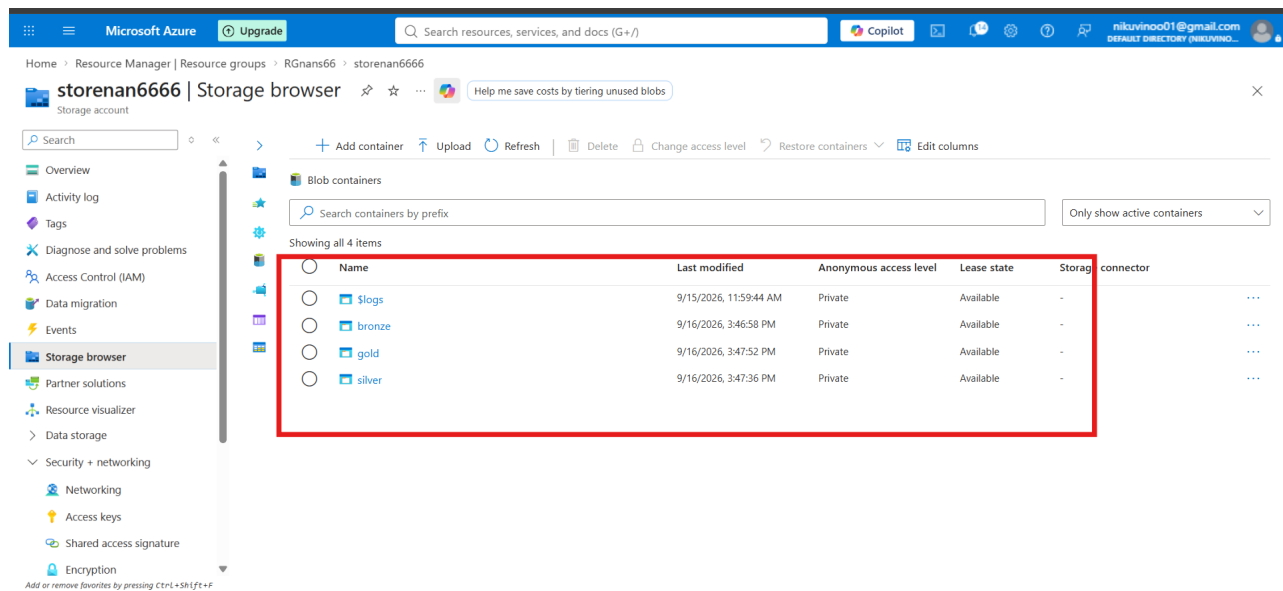


Figure 2 Azure Storage browser showing bronze, silver and gold containers.

2 Bronze source data

The bronze container contains the AdventureWorks Calendar, Customers, Product Categories, Product Subcategories, Products, Returns, Sales 2015, Sales 2016, Sales 2017 and Territories CSV files. The notebook configures access to ADLS and reads source files into Spark DataFrames for inspection.

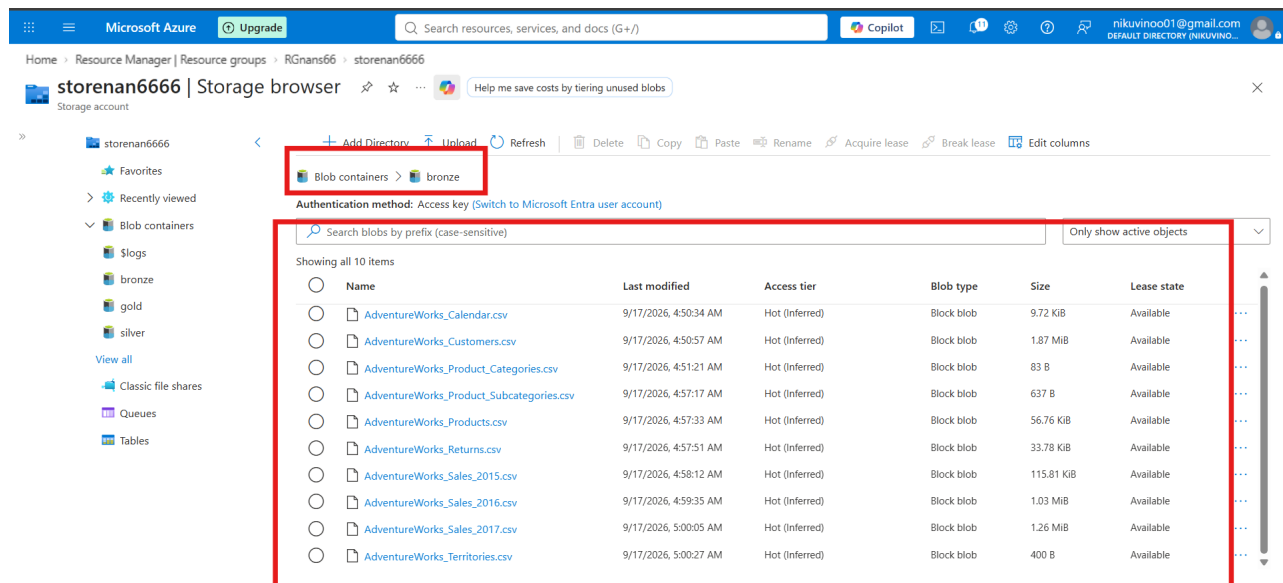


Figure 3 Ten AdventureWorks CSV files in the bronze container.

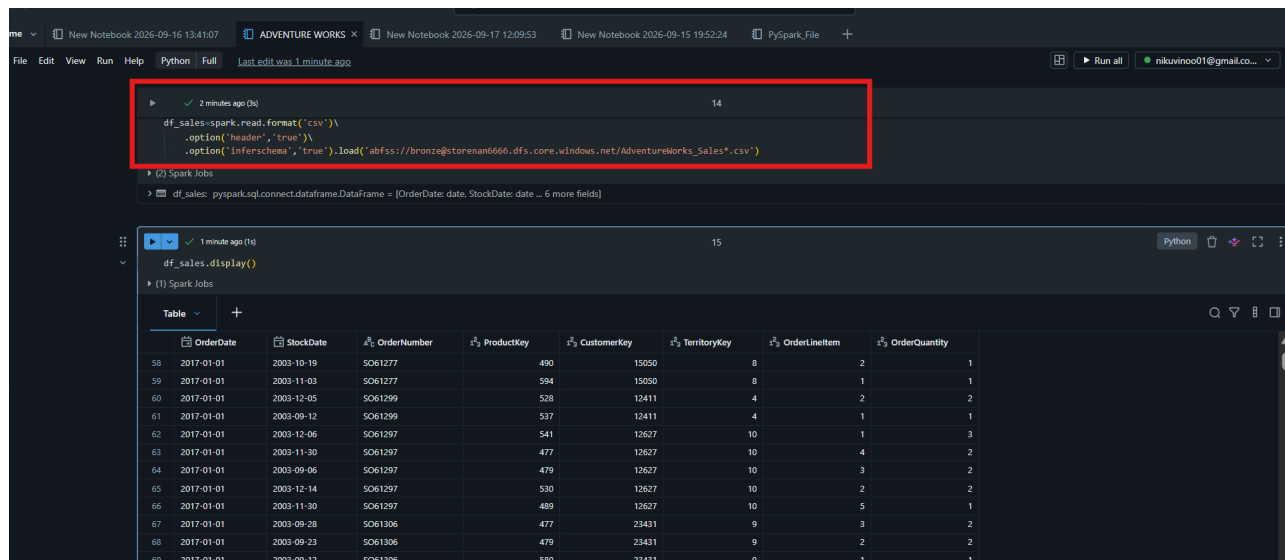


Figure 4 Notebook read and preview of a source dataset.

3 Silver transformations and outputs

Silver processing prepares source data for reliable joins. The notebook previews transformed Calendar, Customer, Product and Sales DataFrames. Visible examples include calendar date attributes, a customer FullName field, product text derived from SKU and product name, and conversion of StockDate to timestamp. The three yearly sales sources are used in the sales preparation. These examples are supported by the notebook images; complete data quality coverage is not established by the screenshots.

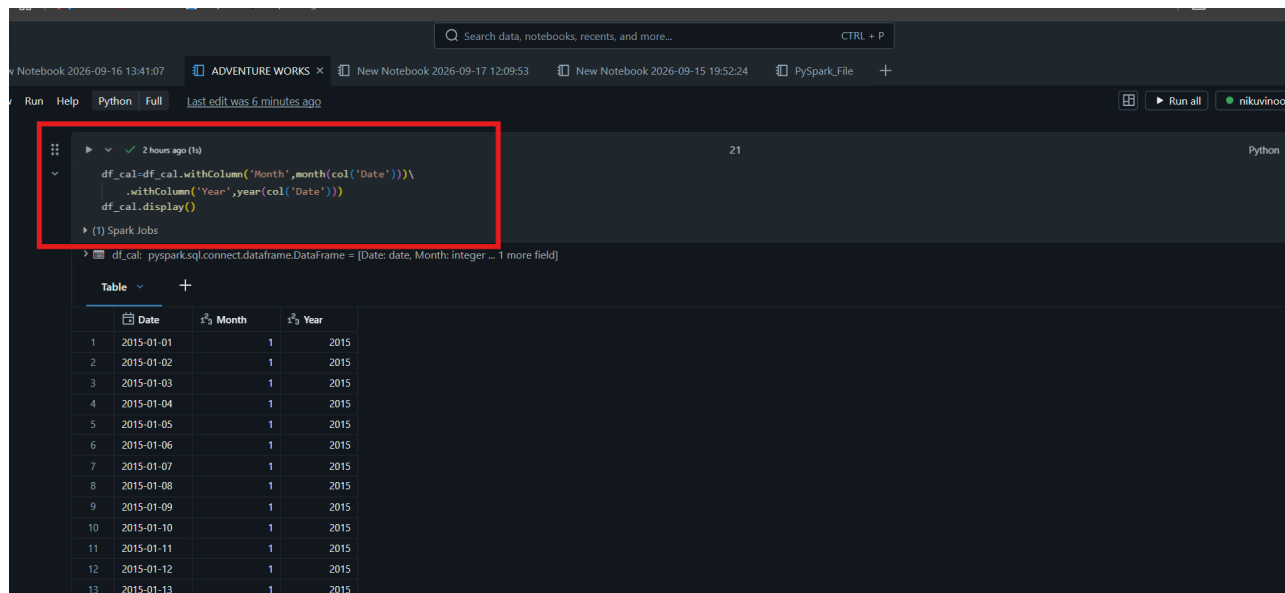


Figure 5 Calendar transformation preview with month and year attributes.

Notebook 2026-09-16 13:41:07 | ADVENTURE WORKS | New Notebook 2026-09-17 12:09:53 | New Notebook 2026-09-15 19:52:24 | PySpark_File

Run Help Python Full Last edit was 9 minutes ago

22

```
from pyspark.sql.functions import col, concat_ws
df_custom = df_custom.withColumn('FullName', concat_ws(' ', col('Prefix'), col('FirstName'), col('LastName')))
df_custom.display()
```

df_custom: pyspark.sql.connect.dataframe.DataFrame = [CustomerKey: integer, Prefix: string ... 12 more fields]

id	name	BirthDate	MaritalStatus	Gender	EmailAddress	AnnualIncome	TotalChildren	EducationLevel	Occupation	HomeOwner	FullName
7		1964-05-09	M	M	marco14@adventure-works.com	\$60,000	3	Bachelors	Professional	Y	M. MARCO MEHTA
8	F	1964-07-07	S	F	rob4@adventure-works.com	\$60,000	4	Bachelors	Professional	Y	MRS. ROBIN VERHOFF
9	N	1964-04-01	S	M	shannon38@adventure-works.com	\$70,000	0	Bachelors	Professional	N	M. SHANNON CARLSON
10		1964-02-06	S	F	jacquelyn20@adventure-works.co...	\$70,000	0	Bachelors	Professional	N	M. JACQUELYN SUAREZ
11		1963-11-04	M	M	curtis9@adventure-works.com	\$60,000	4	Bachelors	Professional	Y	M. CURTIS LU
12		1968-01-18	M	F	lauren41@adventure-works.com	\$100,000	2	Bachelors	Management	Y	MRS. LAUREN WALKER
13		1968-08-06	M	M	ian47@adventure-works.com	\$100,000	2	Bachelors	Management	Y	M. IAN JENKINS
14	F	1968-05-09	S	F	sydney23@adventure-works.com	\$100,000	3	Bachelors	Management	N	MRS. SYDNEY BENNETT
15		1979-02-27	S	F	chloe23@adventure-works.com	\$30,000	0	Partial College	Skilled Manual	N	M. CHLOE YOUNG
16		1979-04-28	M	M	wyatt32@adventure-works.com	\$30,000	0	Partial College	Skilled Manual	Y	M. WYATT HILL
17		1944-06-26	S	F	shannon1@adventure-works.com	\$20,000	4	High School	Skilled Manual	Y	MRS. SHANNON WANG
18		1944-10-09	S	M	clarence32@adventure-works.com	\$30,000	2	Partial College	Clerical	Y	M. CLARENCE RAI
19		1978-03-07	S	M	luke18@adventure-works.com	\$40,000	0	High School	Skilled Manual	N	M. LUKE LAL
20		1978-06-20	S	M	ivordan73@adventure-works.com	\$40,000	0	High School	Skilled Manual	N	M. IVORDAN KING

10,000+ rows Truncated data 1.428s runtime Refreshed 1h ago

Figure 6 Customer transformation preview including FullName.

35 minutes ago

23

```
from pyspark.sql.functions import col, split
df_pro = df_pro.withColumn('ProductPrefix', split(col('ProductSKU'), '-').getItem(0))\
    .withColumn(
        'ProductName_BeforeHyphen',
        trim(split(col('ProductName'), '-').getItem(0)))
display(df_pro)
```

df_pro: pyspark.sql.connect.dataframe.DataFrame = [ProductKey: integer, ProductSubcategoryKey: integer ... 11 more fields]

id	ProductColor	ProductSize	ProductStyle	ProductCost	ProductPrice	ProductPrefix	ProductName_BeforeHyphen
1	Red	0	0	13.0863	34.99	HL	Sport
2	Black	0	0	12.0278	33.6442	HL	Sport
3	White	M	U	3.3963	9.5	SO	Mountain Bike Socks, M
4	White	L	U	3.3963	9.5	SO	Mountain Bike Socks, L
5	Blue	0	0	12.0278	33.6442	HL	Sport
6	Multi	0	U	5.7052	8.6442	CA	AWC Logo Cap
7	Multi	S	U	31.7244	48.0673	LJ	Long
8	Multi	M	U	31.7244	48.0673	LJ	Long
9	Multi	L	U	31.7244	48.0673	LJ	Long
10	Multi	XL	U	31.7244	48.0673	LJ	Long
11	Red	62	U	747.9682	1263.4598	FR	HL Road Frame
12	Red	44	U	747.9682	1263.4598	FR	HL Road Frame
13	Red	48	U	747.9682	1263.4598	FR	HL Road Frame
14	Red	52	U	747.9682	1263.4598	FR	HL Road Frame

293 rows 1.471s runtime Refreshed 35m ago

Figure 7 Product transformation preview including derived product name text.

```
from pyspark.sql.functions import col, to_timestamp, regexp_replace

df_sales = (
    df_sales
    .withColumn(
        "StockDate",
        to_timestamp(col("StockDate"))
    )
    .withColumn(
        "OrderNumber",
        regexp_replace(col("OrderNumber"), "S", "D")
    )
)

display(df_sales)
```

(1) Spark Jobs

df_sales: pyspark.sql.connect.dataframe.DataFrame = [OrderDate: date, StockDate: timestamp ... 6 more fields]

	OrderDate	StockDate	OrderNumber	ProductKey	CustomerKey	TerritoryKey	OrderLineItem	OrderQuantity
1	2017-01-01	2003-12-13T00:00:00.000+00...	DO61285	529	23791	1	2	2
2	2017-01-01	2003-09-24T00:00:00.000+00...	DO61285	214	23791	1	3	1
3	2017-01-01	2003-09-04T00:00:00.000+00...	DO61285	540	23791	1	1	1
4	2017-01-01	2003-09-28T00:00:00.000+00...	DO61301	529	16747	1	2	2
5	2017-01-01	2003-10-21T00:00:00.000+00...	DO61301	377	16747	1	1	1
6	2017-01-01	2003-10-23T00:00:00.000+00...	DO61301	540	16747	1	3	1
7	2017-01-01	2003-09-04T00:00:00.000+00...	DO61269	215	11792	4	1	1

Figure 8 Sales transformation preview including timestamp StockDate.

The notebook write statements use Parquet in the silver container. The Azure Storage view shows eight Silver folders. Append mode is visible in the write screenshot, so a rerun could add files unless the pipeline handles repeats elsewhere.

WRITING TO SILVER

```
df_cal.write.format("parquet").mode("append").save('abfss://silver@storenan6666.dfs.core.windows.net/Adventureworks_cal.parquet')
```

(1) Spark Jobs

```
df_custom.write.format("parquet").mode("append").save('abfss://silver@storenan6666.dfs.core.windows.net/Adventureworks_custom.parquet')
```

(1) Spark Jobs

```
df_cat.write.format("parquet").mode("append").save('abfss://silver@storenan6666.dfs.core.windows.net/Adventureworks_cat.parquet')
```

(1) Spark Jobs

```
df_prosub.write.format("parquet").mode("append").save('abfss://silver@storenan6666.dfs.core.windows.net/Adventureworks_prosub.parquet')
```

(1) Spark Jobs

```
df_pro.write.format("parquet").mode("append").save('abfss://silver@storenan6666.dfs.core.windows.net/Adventureworks_pro.parquet')
```

(1) Spark Jobs

Figure 9 Notebook statements writing transformed DataFrames to Silver Parquet paths.

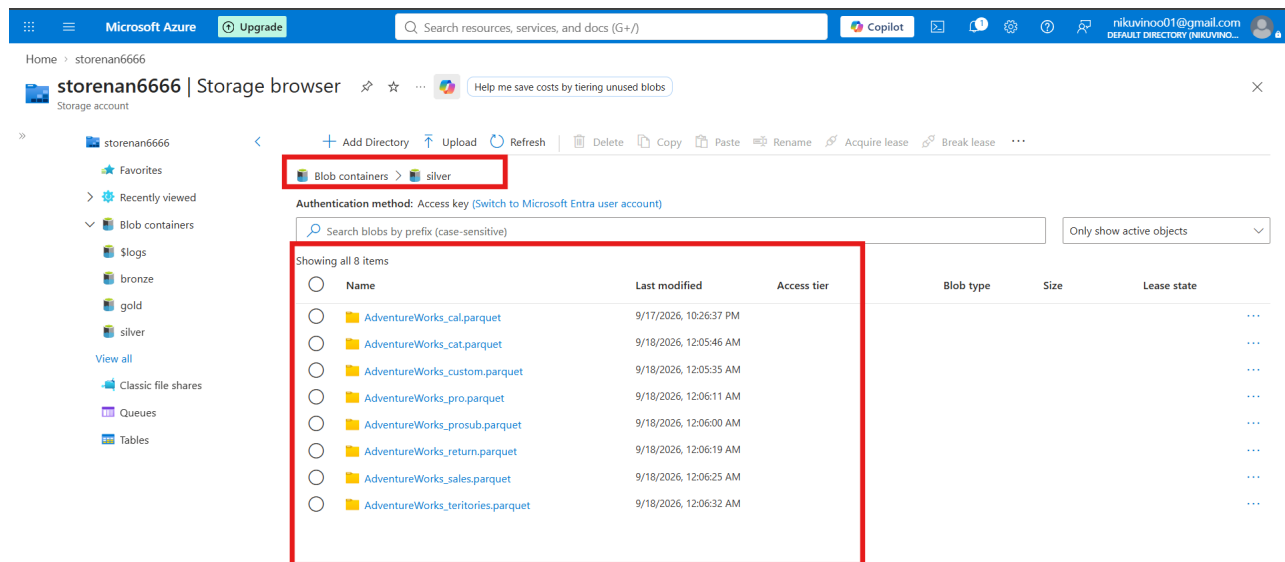


Figure 10 Silver container showing eight Parquet output folders.

4 Gold model and business logic

The Gold layer groups the cleaned records into four dimensions and two facts. The sales fact is modeled at sales order line grain and uses product prices and costs to calculate business measures. Product and related category information provide descriptive context. Returns form a separate fact. This follows the modeling intent of the AdventureWorks guide, while the actual saved table set follows the notebook evidence.

Gold dataset	Role	Observed row count
gold_dim_customer	Customer dimension	18,148
gold_dim_product	Product dimension	293
gold_dim_territory	Territory dimension	10
gold_dim_date	Date dimension	912
gold_fact_sales	Sales fact	56,046
gold_fact_returns	Returns fact	1,809

The screenshots show notebook code for the Returns fact, a quality check for missing product price or cost in Sales, and output counts for all six DataFrames. The missing price or cost check returns 0. These are useful checks, although they do not establish complete key uniqueness or referential integrity for every dimension.

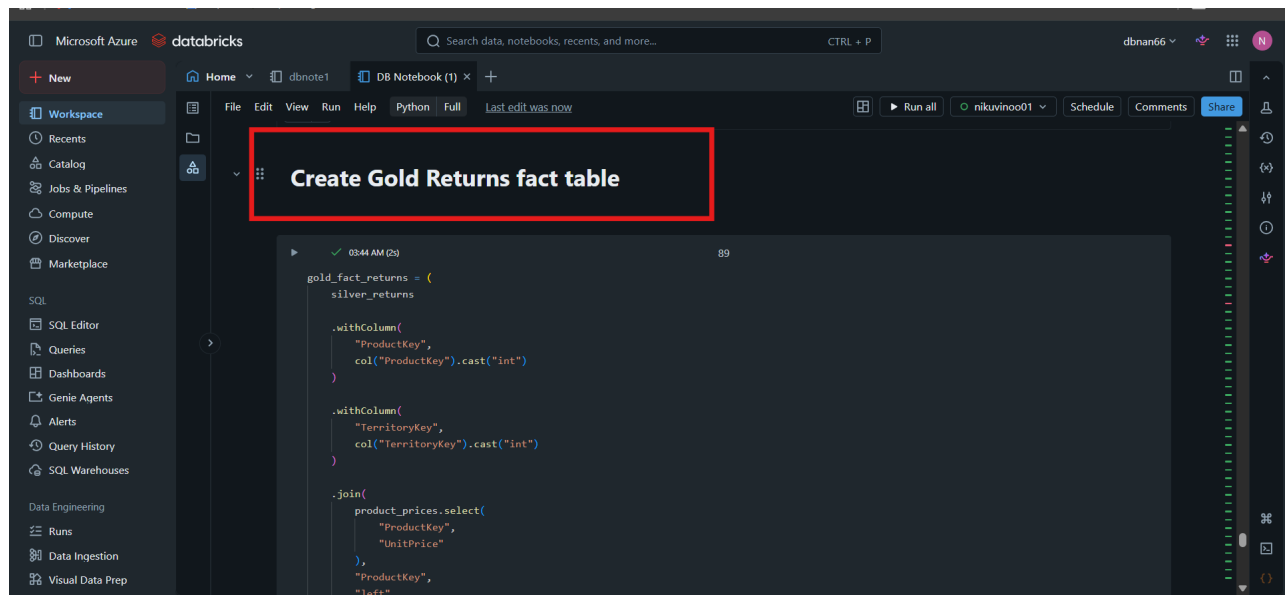


Figure 11 Notebook step creating the Gold Returns fact.

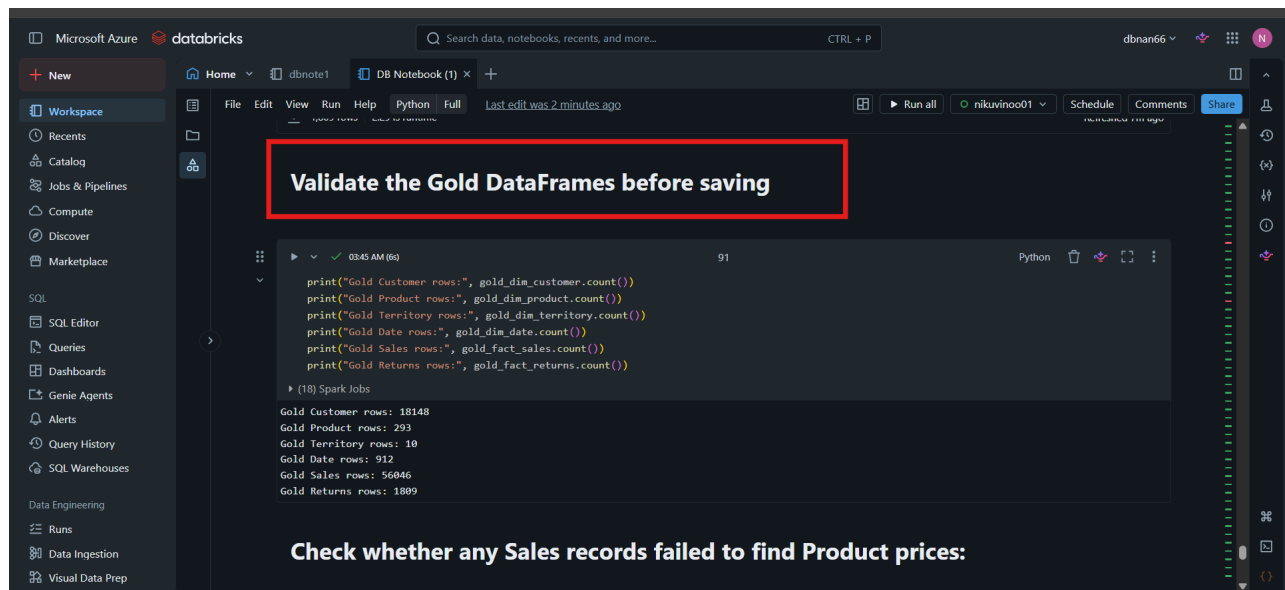


Figure 12 Gold dataset row counts shown by the notebook.

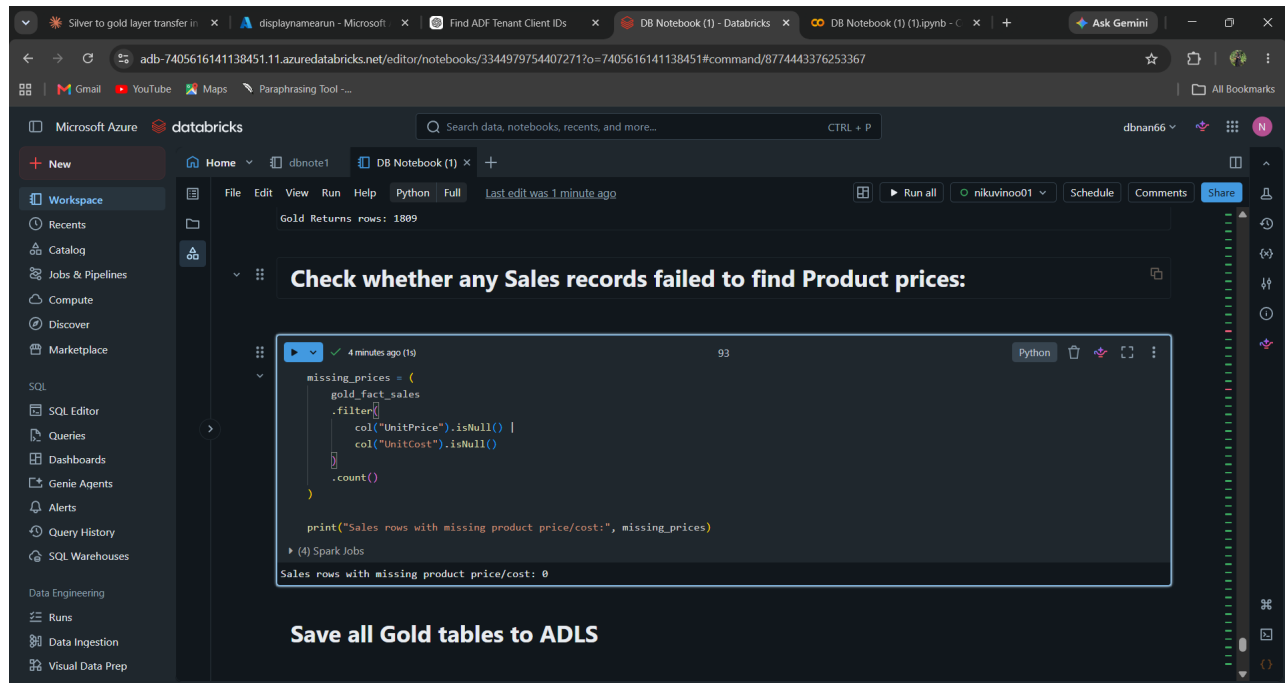


Figure 13 Sales records with missing product price or cost equals zero.

5 Gold storage verification

The notebook writes the six Gold DataFrames with Delta format and overwrite mode to the gold container. A subsequent file listing shows all six folder names. These storage checks support that the Gold data was saved, while Catalog registration is a separate operation and is not shown in the supplied screenshots.

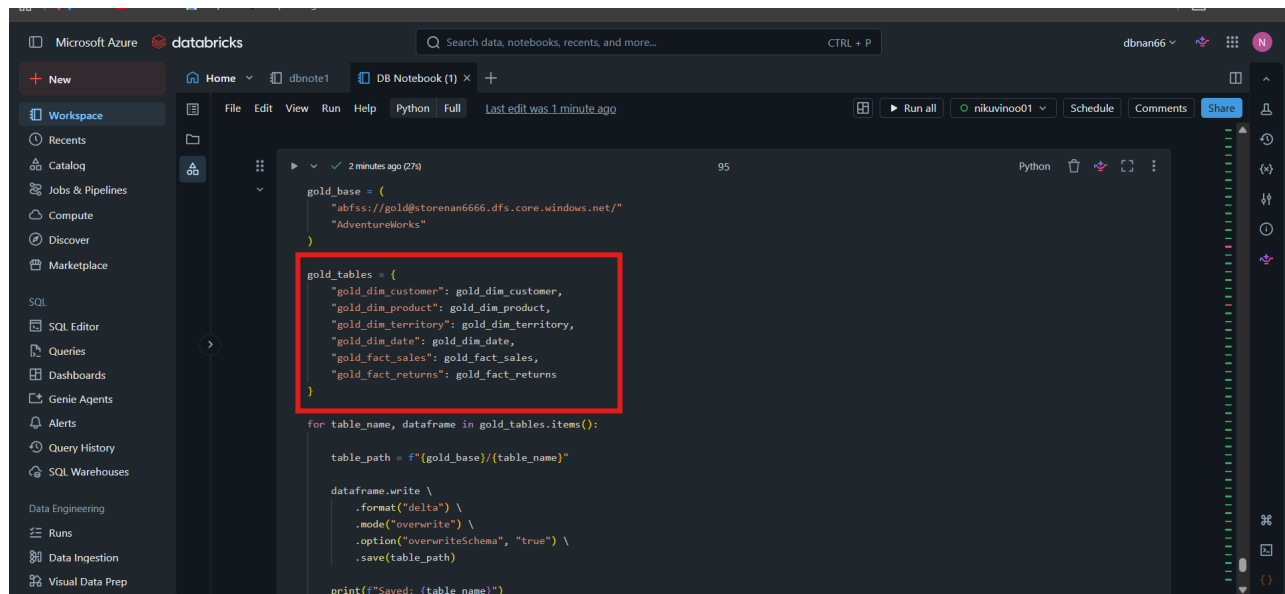


Figure 14 Loop saving all six Gold datasets in Delta format.

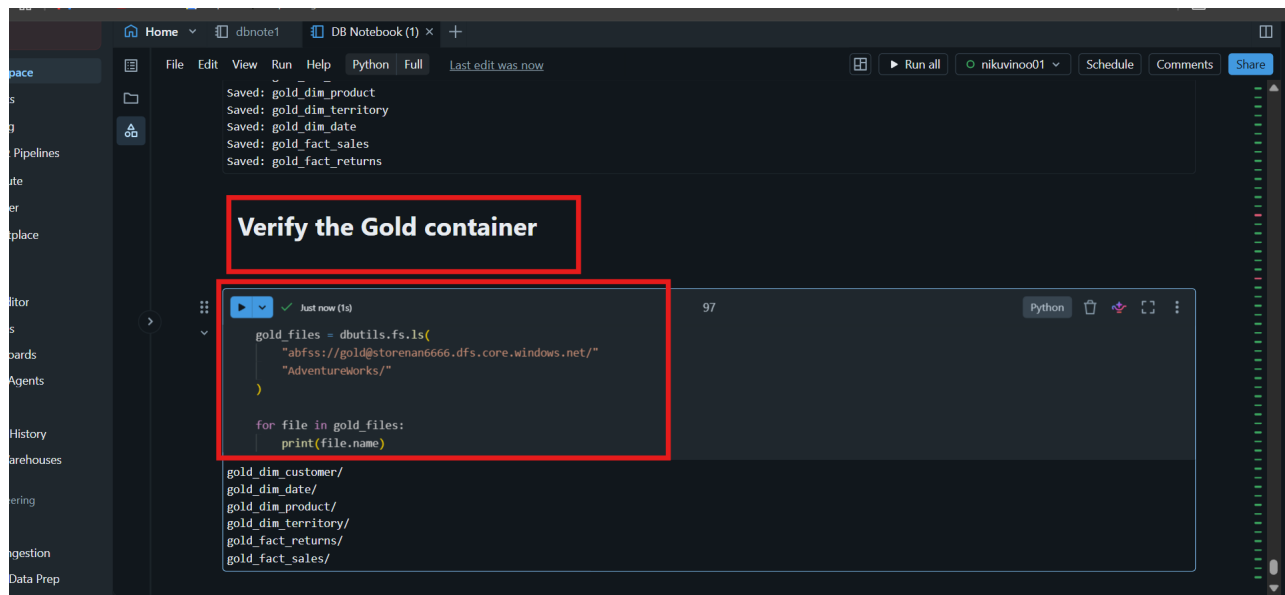


Figure 15 Gold container listing with four dimensions and two facts.

6 Reconstructed Catalog and SQL views

The following screen-style images reconstruct possible Silver and Gold Catalog views and a SQL editor statement from the project details. They are not original screenshots or evidence that Catalog registration or SQL execution occurred. The real screenshots above establish the storage folders and notebook results.

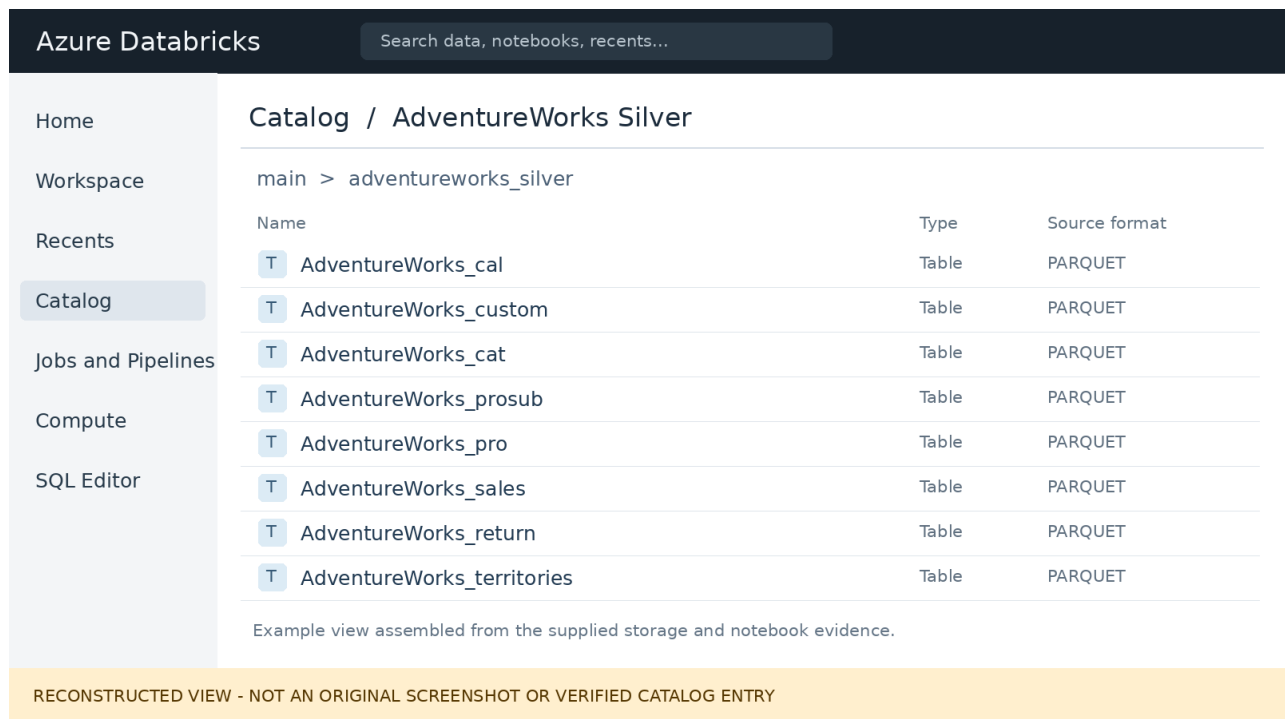


Figure 16 Reconstructed Silver Catalog view based on observed Parquet folders.

Azure Databricks

Search data, notebooks, recents...

Home

Workspace

Recents

Catalog

Jobs and Pipelines

Compute

SQL Editor

Catalog / AdventureWorks Gold

main > adventureworks_gold

Name	Type	Source format
T gold_dim_customer	Table	DELTA
T gold_dim_product	Table	DELTA
T gold_dim_territory	Table	DELTA
T gold_dim_date	Table	DELTA
T gold_fact_sales	Table	DELTA
T gold_fact_returns	Table	DELTA

Example view assembled from the supplied storage and notebook evidence.

RECONSTRUCTED VIEW - NOT AN ORIGINAL SCREENSHOT OR VERIFIED CATALOG ENTRY

Figure 17 Reconstructed Gold Catalog view based on the six Delta folder names.

Azure Databricks

Search data, notebooks, recents...

Home

Workspace

Recents

Catalog

Jobs and Pipelines

Compute

SQL Editor

SQL Editor / Gold table registration

Gold table creation.sql

Run

```

1 CREATE SCHEMA IF NOT EXISTS main.adventureworks_gold;
2
3 CREATE TABLE IF NOT EXISTS
4     main.adventureworks_gold.gold_fact_sales
5 USING DELTA
6 LOCATION 'abfss://gold@storenan6666.dfs.core.windows.net/
7     AdventureWorks/gold_fact_sales';

```

Example statement only. No query result is shown.

RECONSTRUCTED VIEW - NOT AN ORIGINAL SCREENSHOT OR VERIFIED CATALOG ENTRY

Figure 18 Reconstructed SQL editor with an example Gold table registration statement. Not executed.

Project conclusion

The screenshots show ten Bronze CSV files, eight Silver Parquet folders and six Gold Delta folders. Notebook checks report 56,046 Gold Sales rows, 1,809 Gold Returns rows and zero Sales rows missing product price or cost.